

Improve Accessibility of PDF Renderer

GSoC Project proposal for Learning Equality

Synopsis

This project plans to implement a more accessible version of the current Kolibri PDF Renderer. It currently does not support some basic features of a PDF reader such as selecting the text, making annotations, and an outline of the pages, among others. The objective of this project is to update the current PDF renderer plugin with a newer version of the [PDF.js](#) library that allows basic activities typical of a PDF Reader, and customize it so that it maintains consistency with the current Kolibri styles. For this, I will be working on the [Kolibri](#) repository on the [pdf_viewer](#) plugin, with Marcella Maki or Radina Matic as mentors.

Benefits to the Community

PDF files are one of the most used learning resources today, they have a universal format and offer graphical integrity, which makes them very good for transmitting information without modifying its integrity.

On many occasions, PDFs are used to transmit abundant information, and people need to highlight information or put comments to have a lower cognitive load to understand what is written when they read it a second time.

In addition, sometimes there are words that we do not know and we want to find their meaning, so we copy and paste the word immediately into the search engine, or if the text is written in another language, we copy and paste the full text to be able to translate it and understand what it means. Having a PDF that is rendered in a way that you can't interact with words, is a big disadvantage for users, and it's a bad experience for them when they want to copy and paste some long text and they can't, and turns into the user having to transcribe it themselves. This in itself is a big burden for people who type fast, and it is worse for people who are not very used to the keyboard and type slowly, this is something that greatly affects accessibility. Another disadvantage is the inability to read the content by screen readers.

Additionally, it is very helpful to be able to visualize the number of pages that the document contains, and the current page, to get an idea of my progress.

Furthermore, there are people who get too tired of their eyes when they are reading a black text on a white background, so a dark reading mode will surely be very useful for them to avoid tiring their eyes too much, and they can read for much longer.

In conclusion, having a PDF viewer that handles all these concerns would be of great benefit to the community and great support for people to have more facilities when they want to access a reading-type resource in Kolibri.

Current Status of the Project

The current status of the project is a plugin called [pdf_viewer](#) that uses an old version of PDF.js, and it just renders the PDF into a canvas, making it impossible to interact with the text, images, make annotations, etc. For this reason, we need to update the old library to a newer version of PDF.js and take better advantage of its potential to interact with the content of the PDF.

Goals

Goal 1

To render PDFs where users can interact with its textual content, in a way that improves accessibility and allows screen readers to detect the text.

Goal 2

To show the meta information of the PDF and toolbars in a consistent way with Kolibri styles, so that users don't suffer a very abrupt visual change when entering to read a PDF resource.

Goal 3

Provide the ability to create annotations for users to point out important information and make it easier to digest when they read it a second time.

Deliverables

Deliverable 1

The new PDF Viewer with unit tests, consistent styles with Kolibri styles, a pages previewer, page counter, and zoom features like zoom-in, zoom-out, page fit, and page width.

Deliverable 2

A PDF Viewer that allows users to make annotations and save them in the database, also allows them to choose different colors for these annotations, and renders a popup that offers a quick copy and quick search of the text that they have selected.

Deliverable 3

A PDF Viewer that allows users to write notes on annotations, edit notes and delete them, that allows using a dark theme, that remembers the page where the user stayed the last time they opened the PDF, with a word search engine, that has ease of scrolling with the keyboard, and making sure everything is responsive.

Expected Results

The expected result is a PDF Viewer that contains at least the following:

Select text

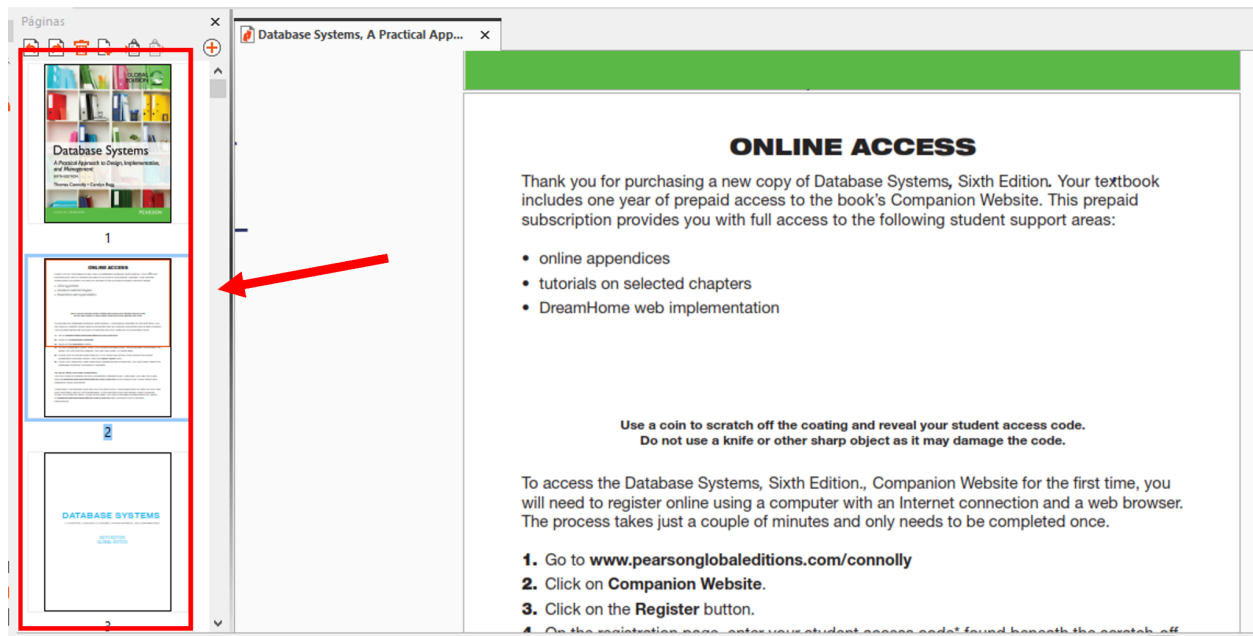
The user is able to select the rendered text of the PDF, and screen readers are able to detect it.

Consistent styles

The styles of the background, buttons, toolbars, and colors are consistent with the general style of Kolibri.

Pages previewer

The rendered PDF is able to show a page previewer where you can see at a high level what the other pages have, something similar to the following:



Page counter

A little card that indicates which page the user is currently on, with an input where users can set the page they want to go to, show how many total pages the PDF has, and a couple of buttons to go to the next and previous page. Something similar to the following:



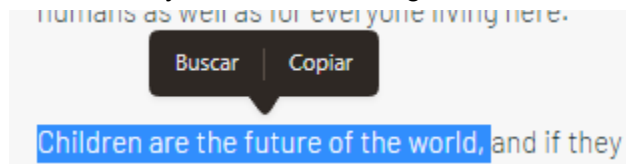
Zoom features

Add buttons in the PDF Viewer that allows to:

- Zoom-in.
- Zoom-out.
- Automatic zoom: Set default zoom.
- Adjust page to height: Set the zoom in a way that the height of a page is the same as the height of the viewport (with some padding).
- Adjust page to width: Set the zoom in a way that the width of a page is the same as the width of the viewport (with some padding).

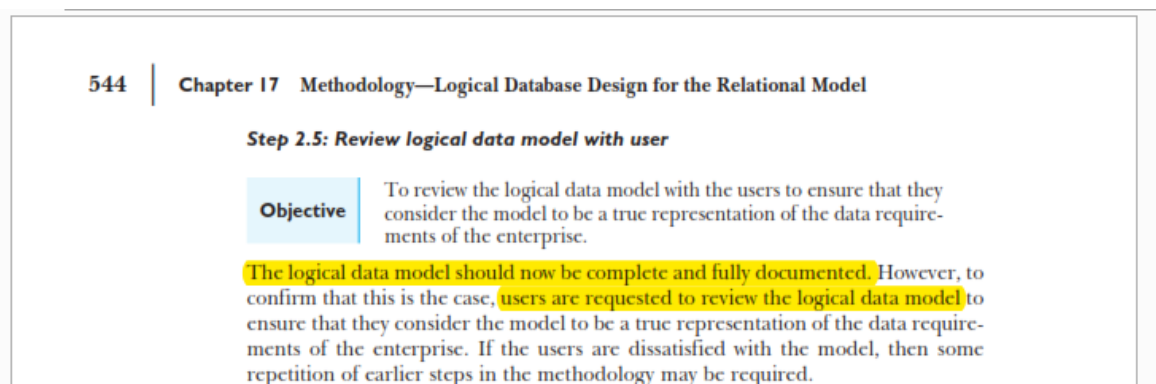
Popup at select text

When selecting text, a popup automatically appears to copy or search in a search engine for the selected text. This is a feature inspired by how Opera handles selected text, and by the time I've been using opera I have found it very useful. Something similar to the following:



Highlight text

Allow users to highlight text, which allows them to highlight ideas, and have a lower cognitive load when they read it again. The user should also have the option to choose the highlighting color. In addition, after highlighting the text, if you right-click on it, you will see a menu that among its options has to delete the highlighted part. Something similar to the following:



Notes

The user can write notes on the highlighted text sections. When they click on the highlighted section, a floating card appears that allows them to write a note, or if they have already done so, the saved note appears. Further, allow users to edit and delete the written note.

Dark mode

I consider this a super necessary accessibility feature for text reading and enabling users to read for much longer without getting tired of reading a black text on a white background.

Last page read remembering

When users enter the PDF a second time, display the last page they were reading when they get out the previous time.

Searcher

Allow users to search for text within the PDF and allow them to navigate between the matches.

Scroll and page changing with arrow keys

Allow scrolling with mouse and keyboard, up and down to scroll vertically, and left and right to switch between pages.

Approach

To accomplish this project I plan to continue using PDF.js to render the PDF into Kolibri updating the library to its latest version. PDF.js is an open-source library that is highly customizable and gives us a lot of control. A drawback is that it doesn't have good documentation, there are some hidden undocumented features, but it has an active community that maintains the library, and they don't take long to respond to issues on GitHub (e.g. a doubt I had, they took less than 30 min to respond to me on [#14791](#)).

Select text

PDF.js allows us to render a PDF with selectable text as shown in a mini example [here](#). What it does is that it renders the PDF in a canvas to keep the original styles without affecting performance (rendering it as SVG is much more expensive), and on top of that, it adds a text layer with the PDF text invisible and in the same position as the original text with HTML tags, which makes it detectable by the browser, and therefore selectable by users and visible to screen readers.

Consistent styles

Unlike other PDF libraries, PDF.js gives us a lot of control over the actions that are done inside the PDF, so the layout of buttons, colors, menus, and toolbars can be set by us following the Kolibri Design System.

Pages previewer

It can be done by putting some thumbnail images of the PDF pages with lazy loading so that the performance is not affected too much. Similar to what is done in the [PDF.js online demo](#) using its [PDF thumbnail viewer](#).

Pages counter

It can be done using the *getPage* method and *numPages* property exposed by PDF.js in its API as shown in one of its [getting started examples](#).

Zoom features

Zoom In and Zoom Out features can be done by scaling the canvas and other layers to specific ratios, and fit width / fit height can be done by setting as width/height the same width/height as the viewport (using vh and vw units). A concern at the moment to make this application I have is how well it will perform scaling HTML layers (like text or annotation layers), but in the [PDF.js online demo](#), the zoom features perform pretty well without affecting the text layer, so it is possible.

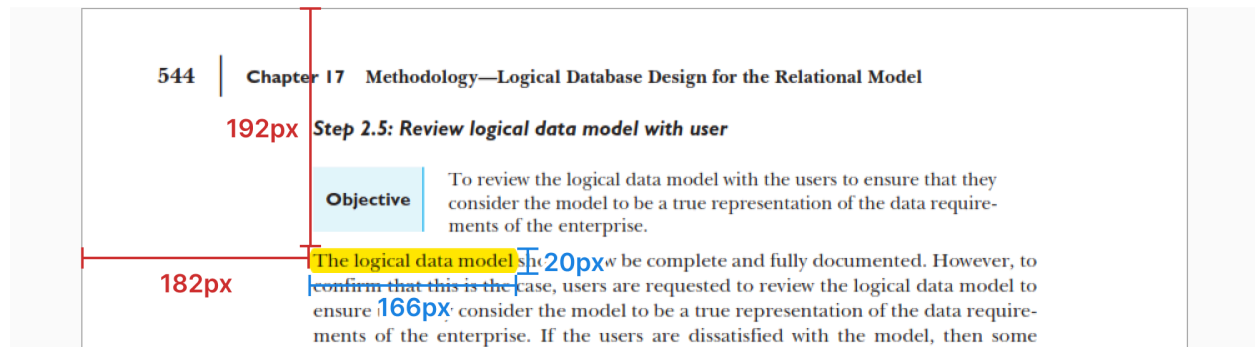
Popup at select text

It can be done through the *selectionchange* event that is emitted when users select, unselect or modify the selection of content on the page. When handling this event, we can call *document.getSelection()* to get the selection object with a lot of information about the selected nodes, their offsets, etc. This object has also a method called *toString()* that returns the selected text string, that we can use to copy or search in the popup actions. To get the position where the popup needs to be rendered, we can use the information returned in the selection object, then math will do the magic to calculate the popup coordinates.

Highlight text

Although PDF.js doesn't support adding annotations like highlight text, it does support reading annotations that the PDF already has, and renders it into an annotation layer. However, this approach of saving the annotations as *objects* into the PDF file is not scalable since we would need a copy of the PDF for every user who would want to add annotations. For this reason, another approach we could have is to save the user annotations as a separate object in the database, and when getting the PDF file, also get this annotations object and render them on top of the canvas as a custom annotation layer.

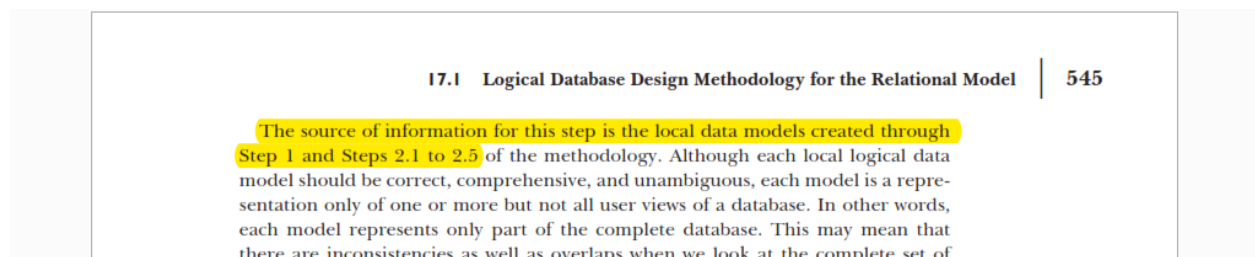
An idea of this annotation object for highlighted text is to have the coordinates X and Y where the top left corner of the annotation is into the canvas, and the height and width of the annotation, so for example, if we have a highlighting like this:



We could have an annotation object like this:

```
{
  selection: {
    x: 182,
    y: 192,
    width: 166,
    height: 20
  },
  type: "highlight"
}
```

But since we could have something like this as a part of the same highlight:



We should treat the selection as an array of rectangles. Adding the customizable color, and taking into account the page number, we could end up with an annotation object like this:

```
{
  page: 1,
  selection: [
    {
      x: 182,
      y: 192,
      width: 166,
      height: 20
    }
  ],
  type: "highlight",
  color: "#FFEA00"
}
```

Note that I added the “type” property for scalability reasons, for example, if in the future, we want to add underline annotations, the same object structure will help us.

Having this, a user should have an array of these annotation objects for a specific PDF resource.

With this object in the Frontend, we just need to place an absolute positioned div in the given coordinates relative to the start of the page, with the given height, width, and background color, and with some opacity and voilà! (actually, it should be a bit more complex, but in essence, that is it).

Notes

To manage the notes, for a given annotation, we could add a click handler to the annotation, show a floating card with input to let the user write the note, or show a saved note, and allow editing and deleting the note, a simple CRUD structure. We could save the note in the database within the annotation object as a property called “note” as a string, or maybe as an object that contains more meta information such as created date, updated date, etc.

To show feedback to the user that the annotation has a note, we should add an icon to the annotated content.

Dark mode

Although PDF.js doesn’t support changing the color of the text and the background to enable a dark mode, we can achieve this through adding a CSS filter to the pages’ canvas that “invert colors” and then play with the filter to get a nice dark mode. A drawback is that the PDF images’ colors will be affected.

Another alternative can be adding the [doq add-on](#) that enables changing the background and text color to different colors getting pretty results, but it has the same drawback that it also applies a CSS filter to the canvas, so the PDF images colors will be affected too.

Last page read remembering

It can be done by saving the current page number into LocalStorage before the user gets out of the PDF viewer, and when a PDF is opened, look at LocalStorage if a page number has been saved, if so, automatically scroll and render that page, if not, render the first page. I suggest saving it in localStorage for speed, but it can be saved in the database too.

Searcher

It can be done using the [pdfFindController](#) that the online demo example of PDF.js uses. It has a lot of additional functionalities that we could adapt, and only use a simpler word searcher to provide a lower load cognitive to users when they want to search some text.

Scroll and page changing with arrow keys

Scrolling up and down with arrow keys is natively supported in most browsers, to get a page change with left and right arrow keys, we could listen to *keydown* events, and if left/right keys are pressed, get the previous/next pages similar to what should be done for page counter feature.

Timeline

This timeline has been made based on my experience and leaving the tasks a little stretched because some small unexpected error can always occur that affects the schedule, and it's more likely that this occurs since the library we are working with it's not well documented. For the tasks of this timeline, time for unit tests is always taken into account even if it is not written in the tasks explicitly.

Period	Task
After proposal submission [April 19 - May 19]	- Wait for GSoC results.

Community bonding [May 20 - June 12]	- Get to know mentors. - Read the documentation in a detailed way. - Acquire more detailed knowledge about the system architecture, the test system, and deployments to production, among others. - Prototype the solution and discuss with mentors how it will look.
Week 1 [June 13 - June 19]	- Update the old version of PDF.js in the PDF Viewer plugin. - Set up unit tests.
Week 2 [June 20 - June 26]	- Style the new PDF Viewer to be consistent with Kolibri styles.
Week 3 - 4 [June 27 - July 11]	- Implement the pages previewer. - Implement the pages counter.
Week 5 - 6 [July 12 - July 24]	- Add zoom features like zoom-in, zoom-out, automatic zoom, adjust the page to height, adjust the page to width.
Week 7 [July 25 - July 31]	- Implement a popup at the select text for quick copy and search actions for the selected text.
Week 8 - 9 [August 1 - August 14]	- Backend support implementing the new annotation object.
Week 10 - 11 [August 15 - August 28]	- Frontend support for highlight text. - Save highlighted text. - Allow deleting text highlighting.

Week 12 [August 29 - September 4]	- Allow users to select the highlighting color.
Week 13 [September 5 - September 11]	- Backend support for adding notes to annotations, create, edit and delete.
Week 14 - 15 [September 12 - September 25]	- Frontend support for adding notes to annotations, create, edit and delete.
Week 16 [September 26 - October 2]	- Implement dark mode.
Week 17 [October 3 - October 9]	- Save into Local Storage on what page the user is on before leaving the PDF. - Check whether the progress system will change for reading resources.
Week 18 - 19 [October 10 - October 23]	- Implement the text searcher.
Week 20 [October 24 - October 30]	- Scrolling with mouse and keyboard, up and down to scroll vertically, and left and right to switch between pages.
Week 21 - 22 [October 31 - November 13]	- Make sure that everything is responsive, and if not, make it responsive. - Documentation. - End of GSOC 2022 :')

If I finish faster or progress faster than planned, some additional features that I could include would be:

- Allow users to have underline annotations.
- Allow users to have crossed annotations.
- Allow users to have floating text annotations.

About Me

I am Alex Velez, an Ecuadorian last year Computer Science student. I consider myself a person with ease of learning, passionate when I love something, and having a great commitment and critical thinking on every project I am part of. I strongly believe that all the effort sooner or later has its reward.

I have a little over a year of experience working with web technologies in some startups, both in backend and frontend (web and mobile), and I have done a little bit of DevOps as well. Although I think I have quite an analytical mindset and a passion for algorithms, I think I also have a great passion for visual details.

Among my web hard skills are Angular, which is the framework I started with and worked with for a few months, React, which is the one I've been working with for about 7 months now, and Vue, which I haven't used in large projects but I do have the basic knowledge by having used it for small university projects. I also have some experience working directly with the DOM, working on mobile apps with Ionic, setting up GitHub Actions, working with some AWS services like EC2, S3, Lambda functions, and Step functions; and I consider myself a bit good with CSS as well.

On the backend, I have experience working with Express with databases like MySQL, HarperDB, Elasticsearch, and MongoDB, and I've also done some work with Flask in Python (more about my most relevant projects and work experience can be found in my portfolio).

One of my greatest achievements is that I together with a wonderful group of friends, was among the 10 global winners of the 2021 Nasa Space Apps Challenge, passing several filters from many evaluators from NASA and other 9 space agencies, and being among the 10 best projects out of 4,500+ globally (I still haven't finished assimilating it). Our project was a prototype of a video game to teach children about the dangers that an astronaut can go through on a long journey through space. This prototype is currently being developed to make it real, and you can read more about it [here](#).

In my experience with Kolibri, I have already finished setting up the project. I had some problems because my OS is Windows, and the installation through docker failed, but I managed to install it through WSL. I have also become a little familiar with the codebase, reading a bit of the documentation. I have contributed with a couple of issues ([#9270](#) and [#9271](#)) and a PR solving one of them ([#9272](#)) that was approved and merged into the *develop* branch and merged into the release branch *release-v0.15.x*. I also tried to fix my first issue but without success.

I chose Learning Equality because I have always been very interested in academics and how technology can help make learning free, more interactive, and more fun. Knowing about

Learning Equality, and its concern to make learning accessible to many people without internet access definitely caught my attention to do my bit in this great project called Kolibri. I've never worked on an Open Source project before, and I think if my first experience was with a group of people who care about helping others it would be really cool. It is also important to say that I will not apply to any other project at GSOC 2022.

During the Google Summer of Code period I will be cursing my 9th semester, but just taking the last 4 subjects of my career, so I will have enough time for the 350 hours in 22 weeks for GSoC. This last year I have been working part-time (20 hrs/week) while cursing 5 subjects, and I was able to carry out both responsibilities satisfactorily, so I have experience handling the pressure.